

Functional Requirements Specification (FRS)

RedBlueAI Safeguard — "RedAgent"

Document	Functional Requirements Specification
Product	RedBlueAI Safeguard (RedAgent) — autonomous, non-destructive red-team validation agent
Version	1.0
Status	Baseline (traceable to implemented Phases 0–10)
Companion docs	docs/SRS.md , docs/HLD_LLD.md , docs/ARCHITECTURE.md , docs/WORKFLOW.md
Owner	ESDS — RedBlueAI Platform

Table of Contents

- 1. Introduction
- 2. System Overview & Actors
- 3. Functional Modules (detailed behaviour)
- 4. Use Cases
- 5. Process Flows
- 6. Interface Behaviour (CLI & API)
- 7. Output / Report Specifications
- 8. Business Rules Catalogue
- 9. Error & Exception Handling
- 10. Traceability & Appendices

1. Introduction

1.1 Purpose

This FRS specifies, in functional detail, **how** RedBlueAI Safeguard behaves for each capability: the inputs it consumes, the processing/decision logic it applies, the outputs it produces, the business rules that govern it, and the error paths. Where the SRS states *what* the system must do (requirement-level), this FRS states *how* the function operates (behaviour-level), so it can be implemented, tested, and verified unambiguously.

1.2 Scope

Functional behaviour of the eleven functional modules: ROE/scope, safety pipeline, recon, scan, planning/orchestration, approval/HITL, safe validation, intelligence, Detection Oracle, reporting, and

continuous/integration — plus the CLI and HTTP interfaces. Non-functional aspects (performance, sovereignty, security posture) are in the SRS §5 and are referenced, not restated.

1.3 Intended Audience

Developers, test engineers (for acceptance-test derivation), security reviewers, and operators.

1.4 Definitions

See `docs/SRS.md` §1.3. Key terms used throughout: **ROE**, **safety class**, **verdict** (`BLOCKED/DETECTED/PARTIAL/MISSED`), **MTTD**, **grounded token**, **PlanDecision**, **ActionOutcome**.

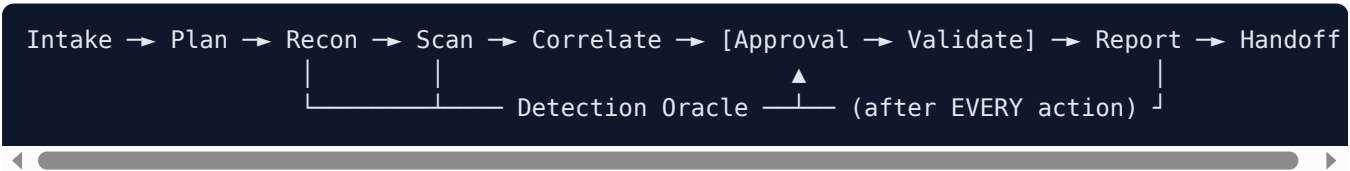
1.5 Requirement Identification

Each function is identified as **FM-x.y** (Functional Module x, function y). Use cases are **UC-n**; business rules are **BR-n**. Acceptance criteria are listed per function as **AC**.

2. System Overview & Actors

2.1 Functional context

Safeguard executes an engagement as a **planner-driven loop** over a checkpointed state machine. The planner proposes the next phase; deterministic code executes it through a single safety pipeline; after every emulated action the Detection Oracle records whether the Blue Team stack noticed; results are enriched, correlated, reported, and handed off to the Detect/Act loops.



2.2 Actors

Actor	Role in functional flows
Operator	Starts engagements (CLI/API), monitors, may kill
Approver (named in ROE)	Signs off / denies active-validate steps
Website/API client	Submits scan jobs, polls results
Planner (Rule or LLM)	Proposes the next <code>PlanDecision</code> — never executes
Detection Oracle	Queries read-only telemetry, scores verdicts
Detect/Act loops	Consume gap report + rule/playbook candidates

3. Functional Modules (detailed behaviour)

FM-1 — Rules-of-Engagement & Scope Enforcement

FM-1.1 Load & validate ROE

- **Trigger:** engagement start (CLI `--roe`, API request, or synthesised ROE).
- **Inputs:** `roe.yaml` (engagement id, owner, authoriser, `authorisation_ref`, mode, profile, scope allowlists, exclusions, timezone, windows, approvers, budget) with `${VAR}` env expansion.
- **Processing:** 1. Parse YAML into typed `RulesOfEngagement`. 2. Validate: `authorisation_ref` present; `profile ∈ {non_destructive}`; ≥1 approver; time windows well-formed (`HH:MM`, 0–23/0–59). 3. On any failure → raise `ValueError`, engagement does not start.
- **Outputs:** an immutable, validated ROE object.
- **Business rules:** **BR-1** fail closed — an ROE missing authorisation, using a non-approved profile, or naming no approver is invalid.
- **AC:** a malformed/under-specified ROE aborts before any tool loads.

FM-1.2 Target scope check

- **Trigger:** every action, before command build; also on-demand via `scope-check`.
- **Inputs:** a `Target(raw, ip?, path?)`; the ROE allowlists/exclusions.
- **Processing (order):** 1. If ROE has no `authorisation_ref` → `OutOfScope`. 2. Decompose URL → `(host, path)`. 3. **Exclusions win:** host/IP in excluded hosts, or path matching an excluded glob → `OutOfScope`. 4. In-scope match: repos/cloud accounts literally; single IP ∈ allowed CIDR; CIDR target only if `subnet_of` an allowed network; domain exact or parent-suffix. 5. No match → `OutOfScope`.
- **Outputs:** pass (silent) or `OutOfScope` (audited, action denied).
- **Business rules:** **BR-2** exclusions override allowlist; **BR-3** a CIDR is in scope only as a subset of an allowed network.
- **AC:** `10.20.30.44` in an allowed /24 → IN SCOPE; `8.8.8.8` → OUT; an excluded host inside an allowed range → OUT.

FM-1.3 Time-window check

- **Inputs:** current time (injected clock) in ROE timezone; window list.
- **Processing:** fail closed if no windows; else match weekday and `start ≤ minutes < end`.
- **Outputs:** pass or `OutOfWindow` (audited, denied).
- **AC:** an action at 03:00 IST inside a 02:00–04:00 window passes; 05:00 is blocked.

FM-2 — Safety Pipeline (the execution choke point)

FM-2.1 Execute one adapter invocation

- **Trigger:** any phase flow needing to run a tool (`SafetyPipeline.execute`).
- **Inputs:** a `ToolAdapter`, an `ActionRequest{invocation, target, actor}`.

- **Processing (strict order; each gate can deny):**

Step	Gate	Denial exception
0	audit <code>tool.proposed</code>	—
1	kill switch engaged?	<code>KillSwitchEngaged</code>
2	scope.check_target	<code>OutOfScope</code>
3	scope.check_window	<code>OutOfWindow</code>
4	class requires_approval & not approved	<code>ApprovalRequired</code>
5	adapter.build_command + validate (forbidden-flag)	<code>SafetyViolation</code>
5b	profile.check (destructive-token denylist)	<code>SafetyViolation</code>
6	rate.acquire (rate + concurrency + budget)	<code>RateLimited</code> / <code>BudgetExceeded</code>
7	runner.run (sandbox)	(execution)
8	adapter.parse → <code>ToolResult</code>	—
9	evidence.put(stdout) → ref	—
10	audit <code>tool.result</code> ; finally rate.release	—

- **Outputs:** `ActionOutcome{result, allowed, denial, audit_head}` . Denials never raise to the caller — they return `allowed=False` and are audited.
- **Business rules:** **BR-4** an adapter is executable **only** here; **BR-5** a kill-switch dominates all other gates; **BR-6** sandbox/parse errors are captured as an error `ToolResult` (still `allowed=True`) and do not abort the engagement.
- **AC:** an out-of-scope invocation produces a `tool.denied` audit event and no execution; a passing invocation produces `tool.proposed` → `tool.exec` → `tool.result` and a content-addressed evidence ref.

FM-2.2 Destructive-action prevention (four layers)

- **Processing:** (1) the `destructive` class is not representable in the `SafetyClass` enum, so `tools.yaml` declaring it fails to load; (2) per-tool `forbidden_flags` ; (3) per-adapter mode ceilings; (4) global `ProfileGuard` token denylist checked on every built command.
- **BR-7:** only the `non_destructive` profile enables execution; all else fails closed.
- **AC:** an attempt to pass `--dump` / `--os-shell` to SQLMap, or `dos` / `intrusive` tags to Nuclei, is rejected at build/validate time.

FM-2.3 Rate / blast-radius limiting

- **Inputs:** target key, monotonic time, ROE budget (rps, concurrency, total).
- **Processing:** token-bucket refill by `elapsed×rate`; deny if in-flight ≥ cap, tokens < 1, or total actions ≥ ceiling — **without mutating state on denial**; `acquire` increments in-flight + total, `release` decrements in-flight.
- **AC:** the (N+1)-th concurrent action to one target beyond the cap is `RateLimited` ; global budget exhaustion is `BudgetExceeded` .

FM-2.4 Kill switch

- **Processing:** `engage()` sets a flag and fires registered revocation hooks (wired to `runner.revoke()`); subsequent runs raise `SandboxError`.
 - **AC:** after kill, no new tool executes and in-flight runners are revoked.
-

FM-3 — Recon & Asset Discovery

FM-3.1 Run recon plan

- **Trigger:** planner selects `recon` (black/gray-box, no assets yet).
- **Inputs:** in-scope targets; plan (default `nmap → httpx → whatweb`; optional `gobuster` with wordlist); per-tool params.
- **Processing:** for each tool in plan, invoke via the safety pipeline; parse output into `Asset` s; record step status (allowed/denied/error, counts).
- **Outputs:** a recon report (per-step results + merged inventory + allowed/denied counts).
- **BR-8:** a denied/errored step is recorded and the flow continues (non-fatal).

FM-3.2 Consolidate assets

- **Processing:** `AssetInventory` dedup/merge on `(type, address, port, protocol)`; deep-merge tech fingerprints across tools; index `by_type`, `hosts()`.
 - **AC:** two tools reporting the same host:port collapse to one asset carrying the union of tech fingerprints.
-

FM-4 — Vulnerability Detection (Scan)

FM-4.1 Run mode-aware scan

- **Trigger:** planner selects `scan`.
- **Inputs:** targets + plan derived from mode: black-box `nuclei,nikto` over endpoints; gray-box `prowler,trivy` over cloud accounts; white-box `semgrep,gitleaks,checkov,trivy` over repos.
- **Processing:** invoke each tool via the pipeline; parse into `Finding` s (severity, CVE/CVSS where available).
- **Outputs:** a merged `FindingLedger`.
- **BR-9:** Nuclei runs safe-only (intrusive/DoS/fuzz/brute tags excluded; re-enabling blocked). **BR-10:** Gitleaks never stores the secret value. **BR-11:** Prowler is read-only (mutating flags rejected).

FM-4.2 Consolidate findings

- **Processing:** `FindingLedger` dedup on `(asset_ref, title.lower)`; keep highest severity; union CVEs/evidence/techniques; record contributing tools in `raw['sources']`; expose `by_severity()`.
 - **AC:** the same issue found by two tools appears once, at the higher severity, crediting both tools.
-

FM-5 — Planning & Orchestration

FM-5.1 Decide next phase

- **Trigger:** the graph re-enters the planner after every phase.
- **Inputs:** `AgentState` (inventory, ledger, plan history, budget, mode, profile).
- **Processing (RulePlanner default):** 1. budget reached → `report` ; 2. black/gray-box & no assets & not done recon → `recon` ; 3. no findings & not done scan → `scan` ; 4. findings & not done correlate → `correlate` ; 5. profile non-destructive, not white-box, not done validate, a HIGH/CRITICAL finding exists → `validate` (dalfox, `requires_approval=true`) ; 6. else → `report` .
- **Processing (LLMPlanner):** send grounded state summary + allowed tools to Qwen; parse JSON `PlanDecision` ; **validate in code** — unknown action → `report` ; a `validate` proposal whose tool is not a registered `active-validate` → `report` ; anti-loop (don't repeat one-shot stages) and anti-premature-report guards; on LLM error/malformed JSON → deterministic fallback.
- **Outputs:** a `PlanDecision{action, tool?, target?, technique?, rationale, requires_approval, params}` .
- **BR-12:** the planner only proposes; it never executes. **BR-13:** the LLM cannot widen its own authority (unregistered/destructive proposals are downgraded).

FM-5.2 Route & execute the decision

- **Processing:** conditional edges map action → node (`recon/scan/correlate/ approval_gate/report`); passive nodes run their flow then loop back to planner; `validate` routes through `approval_gate` .
- **AC:** a passive engagement autonomously sequences `recon` → `scan` → `correlate` → `report` with no human input.

FM-5.3 Checkpoint & resume

- **Processing:** state is serialised to a dict and persisted (SQLite/in-memory) after every node; a run can resume from the last checkpoint.
- **AC:** a run interrupted for approval resumes at the exact parked node after sign-off; a completed run is replayable from its checkpoint history.

FM-6 — Human-in-the-Loop Approval

FM-6.1 Park for approval

- **Trigger:** planner returns an `active-validate` decision → `approval_gate` node.
- **Processing:** create an `ApprovalRequest{request_id, tool, target, technique, rationale}` (status `pending`), audit `approval.requested` , and raise `GraphInterrupt` — the run **parks** and is checkpointed.
- **Outputs:** `state.pending_approval` ; the run status becomes `interrupted` .

FM-6.2 Resolve approval

- **Trigger:** operator calls `approve(request_id, approver, decision)` .

- **Processing:** verify the approver is a **named ROE approver** (else `PermissionError`); record decision in the store.
- **BR-14:** only a named approver may sign off; **BR-15:** denial or a still-pending gate routes back to the planner (the step is skipped).

FM-6.3 Resume

- **Processing:** on resume the gate re-reads the store: `approved` → `validate` node; `pending` → re-interrupt (keep parked); `denied` → back to planner. Audit `approval.resolved`.
 - **AC:** approve → validation runs; deny → validation skipped, engagement completes.
-

FM-7 — Safe Validation

FM-7.1 Run one approved validation

- **Trigger:** `validate` node after approval.
 - **Inputs:** the approved tool+target+technique; approver identity.
 - **Processing:** run the active-validate adapter (Dalfox reflection-only marker; SQLMap `--technique=BT`, low level/risk, `--batch`) through the safety pipeline; capture content-addressed evidence; stamp the approver onto each `Validation`; then invoke the Detection Oracle for the attempt.
 - **Outputs:** `Validation{target, method, result (confirmed/inconclusive), tool, approved_by, evidence_ref, non_destructive=true}`.
 - **BR-16:** validations are non-destructive by construction — no dump/shell/exfil/ DoS/persistence; enforced by profile + forbidden_flags + adapter ceiling, not by prompt.
 - **AC:** a validation without approval never runs; an approved reflected-XSS check confirms the signal and stores evidence; no destructive flag is reachable.
-

FM-8 — Intelligence & Correlation

FM-8.1 Enrich findings

- **Trigger:** `correlate` node.
- **Processing:** attach CVE detail from the **local NVD mirror** (no external call), map to ATT&CK techniques (local keyword map), compute risk = CVSS + EPSS + asset criticality + **detection status**; return the grounded-token set (numbers sourced only from artifacts).
- **BR-17:** an undetected medium can outrank a detected high (detection-aware risk).

FM-8.2 Correlate attack paths

- **Processing:** group findings per asset root, order steps by ATT&CK tactic rank then risk, annotate each step with its detection verdict (`UNKNOWN` until the Oracle runs), path risk = max step risk; sort paths by risk.
- **Outputs:** candidate `AttackPath`s (hypotheses; nothing is executed).

FM-8.3 Ground numeric claims

- **Processing:** `NumericClaimVerifier` flags any CVE token in report narrative not present in the grounded set.
 - **BR-18:** no ungrounded CVE enters a report unflagged.
-

FM-9 — Detection Oracle ★

FM-9.1 Observe an action

- **Trigger:** automatically after every recon/scan/validate action (`_observe`).
- **Inputs:** `action_ref`, `target`, `technique`, `action_time`; correlation window (default 300 s).
- **Processing:** build window `[t-5s, t+window]` ; query every read-only connector (Wazuh/WAF/EDR/PAM/DAM); score a single verdict = best across sources (`BLOCKED > DETECTED > PARTIAL > MISSED`) and MTTD = earliest detection – action time; append `DetectionResult` ; update `detection_status[root(target)]` .
- **Outputs:** a `DetectionResult` per target; the detection-status map feeding risk.
- **BR-19:** connectors are read-only by construction; only WAF/EDR may return `BLOCKED` . **BR-20:** events outside the window are ignored.
- **AC:** empty telemetry → `MISSED` , 0 % coverage, gap report populated; a WAF block outranks a SIEM detect.

FM-9.2 Aggregate coverage

- **Processing:** `CoverageMatrix` computes coverage %, mean MTTD, per-technique matrix, and the gap report (every MISSED/PARTIAL + expected detection).
 - **Outputs:** `detection_coverage` summary in the report.
-

FM-10 — Reporting & Evidence

FM-10.1 Build report bundle

- **Trigger:** `report` node on completion.
- **Processing:** assemble `report.json` (assets, hosts, findings, severity counts, attack paths, top risk, validations, detection coverage, actions spent, numeric verification) + a `detection_index` (technique|host → verdict); render executive summary, technical report, ATT&CK heatmap, and detection-gap report.
- **Outputs:** files under `runs/<id>/report/` .
- **BR-21:** narrative figures are grounded; the executive summary flags any ungrounded figure.

FM-10.2 Capture evidence

- **Processing:** raw tool stdout stored as SHA-256 content-addressed files; findings/validations reference it by hash.
 - **BR-22:** the same output stored twice yields one file (deduplication by content).
-

FM-11 — Continuous Mode & Platform Integration

FM-11.1 Baseline diff / regression detection

- **Trigger:** engagement completion (`ContinuousRunner.record`).
- **Processing:** diff fresh bundle vs stored baseline on `detection_index` : **regression** = key covered (DETECTED/BLOCKED) before, MISSED/PARTIAL now; also improvements, new/resolved gaps & findings, coverage delta; then save the fresh bundle as the new baseline.
- **Outputs:** a `RegressionReport` .
- **BR-23:** a technique/host that was detected last run and is now missed is the headline regression signal.

FM-11.2 Detect/Act handoff

- **Processing:** each gap → `RuleCandidate` (technique, target, expected detection, source wazuh|waf, priority); response `playbooks` + Jira-style `tickets` ; write JSON to `runs/<id>/handoff/` .
- **Outputs:** `detect_rule_candidates.json` , `act_playbooks.json` , `act_tickets.json` .

FM-12 — Control Plane & Governance

FM-12.1 RBAC-gated actions

- **Processing:** roles `admin/operator/approver/viewer` × actions `start/approve/kill/query` ; `require()` raises `AccessDenied` when disallowed; approval additionally requires named-approver status.
- **BR-24:** least privilege — an approver may sign off but not start; a viewer may only query.

FM-12.2 Audit & verify

- **Processing:** every plan/proposal/approval/exec/result/denial appended as a hash-chained `AuditEvent` ; `audit-verify` recomputes the on-disk chain.
- **BR-25:** tampering with any record breaks the chain and is detected.

4. Use Cases

UC-1 — Black-box engagement (CLI, planner-driven)

1. Operator runs `safeguard engage --roe roe.yaml` .
2. Intake validates scope; planner → recon; Nmap/httpx/WhatWeb build the asset inventory; Oracle checks recon detection.
3. Planner → scan; Nuclei/Nikto produce findings; Oracle checks scan detection.
4. Planner → correlate; CVE/ATT&CK/risk enrichment + attack paths.
5. Planner → validate (HIGH finding) → **parks** for approval.
6. Operator re-runs with `--approve <name>` ; Dalfox confirms reflected XSS; Oracle → MISSED (gap).
7. Report bundle + continuous diff + Detect/Act handoff written. - **Value delivered:** not "the app has XSS" but "the app has XSS and your WAF and SIEM both missed it — here is the rule to add".

UC-2 — White-box source assessment (API)

1. Client `POST /api/v1/scans {mode: white_box, repos: [/srv/app]}`.
2. Service synthesises a non-destructive ROE; planner **skips network recon**.
3. Scan runs Semgrep/Gitleaks/Checkov/Trivy over the source.
4. Findings enriched, reported; job result returned on poll.

UC-3 — Gray-box cloud posture (API)

1. Client `POST /api/v1/scans {mode: gray_box, cloud_accounts: [...]}`.
2. Prowler/Trivy (read-only) assess configuration; findings folded into the same pipeline and report.

UC-4 — Emergency stop

1. During a run the operator engages the kill switch.
2. In-flight sandbox runners are revoked; no further tool executes; state remains checkpointed and auditable.

UC-5 — Audit verification

1. Reviewer runs `safeguard audit-verify --roe roe.yaml`.
2. System recomputes the hash chain and reports INTACT/BROKEN + event count.

UC-6 — Continuous regression watch

1. Scheduled re-run of the same engagement.
2. System diffs vs baseline; reports any technique that regressed DETECTED → MISSED and coverage delta.

5. Process Flows

5.1 Safety-gated tool execution

```
PlanDecision → phase flow → SafetyPipeline.execute
  kill? scope? window? approval? build/validate? profile? rate?
    | any deny → audit tool.denied, ActionOutcome(allowed=False)
    ▼ all pass
  sandbox.run → parse → evidence.put → audit tool.result → ActionOutcome
```

5.2 HITL approval (sequence)

[illegible]

5.3 Every-action detection loop

```
recon/scan/validate → _observe(action_ref, targets, technique, action_time)
  ► query Wazuh/WAF/EDR/PAM/DAM over [t-5s, t+window]
  ► score best verdict + MTTD ► append DetectionResult ► update detection_status
```

6. Interface Behaviour

6.1 CLI

Command	Function	Key outputs
<code>run --phase recon</code>	FM-3 recon over in-scope targets	asset inventory, audit head
<code>run --phase scan</code>	FM-4 scan	findings by severity, audit head
<code>engage</code>	FM-5 full planner-driven loop	plan history, report, coverage %, handoff
<code>engage --approve <name></code>	resume a parked validation (FM-6)	validation + resumed report
<code>scope-check <target></code>	FM-1.2 scope decision	IN/OUT/EXCLUDED
<code>audit-verify</code>	FM-12.2 chain verification	INTACT/BROKEN + count

6.2 HTTP API (async jobs)

Endpoint	Method	Behaviour
<code>/api/v1/modes</code>	GET	self-describing per-mode input contract (drives UI form)
<code>/api/v1/scans</code>	POST	validate inputs (fail closed) → queue job → <code>202 {job_id}</code>
<code>/api/v1/scans/{id}</code>	GET	job status; on completion, summary + report paths + planner info
<code>/api/v1/scans/{id}/report</code>	GET	full <code>report.json</code> bundle
<code>/health</code>	GET	liveness + available modes

- **BR-26:** unknown request fields → 400; missing mode-specific inputs → 422.
- **BR-27:** a zero-finding scan caused by a missing scanner binary is reported via `unavailable_tools`, not presented as a clean target.
- **BR-28:** the synthesised ROE is still scope-guarded — the API does not trust the caller for authorisation beyond the scope it generates.

7. Output / Report Specifications

Artifact	Contents
<code>report.json</code>	assets, hosts, findings, severity_counts, attack_paths, top_risk, validations, detection_coverage, actions_spent, numeric_verification, detection_index
<code>executive_summary.md</code>	posture, top risks, coverage %, flagged ungrounded figures
<code>technical_report.md</code>	findings + evidence + severity + reproduction
<code>attack_heatmap.md</code>	ATT&CK technique × verdict + technique coverage %
<code>detection_gap_report.md</code>	every MISSED/PARTIAL + expected-but-absent detection
<code>detect_rule_candidates.json</code>	SIEM/WAF rule candidates per gap
<code>act_playbooks.json</code> / <code>act_tickets.json</code>	candidate response playbooks + tickets
<code>audit.log.jsonl</code>	hash-chained event stream
<code>evidence/ev-<hash>.txt</code>	content-addressed raw tool output
<code>baselines/baseline-NNNN.json</code>	prior bundles for regression diffing

8. Business Rules Catalogue (consolidated)

ID	Rule
BR-1	ROE without authorisation/approver or with non-approved profile is invalid (fail closed).
BR-2	Exclusions override the allowlist unconditionally.
BR-3	A CIDR target is in scope only as a subset of an allowed network.
BR-4	An adapter executes only from the safety pipeline.
BR-5	The kill switch dominates all other gates.
BR-6	Sandbox/parse errors are captured, not fatal.
BR-7	Only the <code>non_destructive</code> profile enables execution.
BR-8	A denied/errored recon/scan step is non-fatal.
BR-9	Nuclei is safe-only; intrusive tags cannot be re-enabled.
BR-10	Gitleaks never stores the secret value.
BR-11	Prowler is read-only.
BR-12	The planner proposes; it never executes.
BR-13	The LLM cannot widen its own authority.
BR-14	Only a named ROE approver may sign off active steps.
BR-15	Denial/timeout skips the step and returns to the planner.
BR-16	Validations are non-destructive by construction.
BR-17	Risk is detection-aware — undetected can outrank detected.
BR-18	No ungrounded CVE enters a report unflagged.
BR-19	Detection connectors are read-only; only WAF/EDR may BLOCK.
BR-20	Detection events outside the correlation window are ignored.
BR-21	Report figures are grounded to artifacts.
BR-22	Evidence is content-addressed (dedup by content).
BR-23	A DETECTED → MISSED change is the headline regression.
BR-24	Control-plane roles are least-privilege.
BR-25	Audit tampering is detectable via the hash chain.
BR-26	Unknown/missing API fields fail closed (400/422).
BR-27	Missing scanner binaries are reported, not hidden.
BR-28	The API-synthesised ROE is still scope-guarded.

9. Error & Exception Handling

Condition	Function	Handling
Out-of-scope target	FM-1.2	<code>OutOfScope</code> → audit <code>tool.denied</code> , <code>allowed=False</code> , no execution
Out-of-window	FM-1.3	<code>OutOfWindow</code> → denied + audited
Missing approval	FM-2.1/6	<code>ApprovalRequired</code> → step parked or skipped
Forbidden flag / destructive token	FM-2.2	denied at build/validate/profile
Rate/budget exceeded	FM-2.3	<code>RateLimited</code> / <code>BudgetExceeded</code> , state unmutated
Kill engaged	FM-2.4	<code>KillSwitchEngaged</code> / <code>SandboxError</code> , runners revoked
Tool binary absent	FM-3/4	<code>SandboxError</code> ; API surfaces <code>unavailable_tools</code>
Sandbox/parse failure	FM-2.1	captured as error <code>ToolResult</code> , engagement continues
LLM error / malformed JSON	FM-5.1	fallback to deterministic RulePlanner (recorded)
Non-named approver	FM-6.2	<code>PermissionError</code> , not resolved
Invalid/under-specified ROE	FM-1.1	<code>ValueError</code> , engagement does not start

10. Traceability & Appendices

10.1 FM ↔ SRS ↔ module

FM	SRS features	Primary modules
FM-1	4.1	<code>config/</code> , <code>safety/scope_guard.py</code>
FM-2	4.2	<code>safety/pipeline.py</code> , <code>rate_limiter</code> , <code>killswitch</code> , <code>profile</code>
FM-3	4.3	<code>recon/</code> , adapters <code>nmap/httpx/whatweb/gobuster</code>
FM-4	4.4	<code>scan/</code> , adapters <code>nuclei/nikto/trivy/prowler/semgrep/gitleaks/checkov</code>
FM-5	4.5	<code>graph/</code> , <code>llm/planner.py</code> , <code>orchestrator.py</code>
FM-6	4.6	<code>safety/approvals.py</code> , <code>graph/build.py</code>
FM-7	4.7	<code>validate/flow.py</code> , adapters <code>dalfox/sqlmap</code> , <code>evidence.py</code>
FM-8	4.8	<code>intel/</code>
FM-9	4.9	<code>oracle/</code>
FM-10	4.10	<code>reporting/</code> , <code>evidence.py</code>
FM-11	4.11	<code>continuous/</code> , <code>integration/</code>
FM-12	4.12–4.13	<code>api/</code> , <code>safety/rbac.py</code> , <code>safety/audit.py</code>

10.2 Safety classes & gates

Class	Gate	Tools
passive	rate limit	<code>cve_lookup</code> , <code>semgrep</code> , <code>gitleaks</code> , <code>checkov</code>
active-recon	rate limit	<code>nmap</code> , <code>httpx</code> , <code>whatweb</code> , <code>gobuster</code> , <code>nuclei</code> , <code>nikto</code> , <code>trivy</code> , <code>prowler</code>
active-validate	approval + rate limit	<code>dalfox</code> , <code>sqlmap</code>
destructive	not loadable / unreachable	—

10.3 Verdict precedence

BLOCKED > DETECTED > PARTIAL > MISSED

End of FRS. Behaviour described here is traceable to the implemented codebase (Phases 0–10); items dependent on production backends (egress-pinned sandbox, live telemetry, LLM planner) behave as specified once the corresponding interface is wired, per `IMPLEMENTATION.md` .